

Практическая работа 2. Особенности написания программ на CUDA

Теперь, когда мы познакомились с базовыми понятиями CUDA и написали свою первую программу, перейдем к изучению основ работы с памятью на устройстве, узнаем, каким образом осуществляется взаимодействие CPU и GPU, как отслеживаются ошибки, которые неизбежно возникают при написании кода и, наконец, узнаем о способах синхронизации нитей, предусмотренных в CUDA. Также напишем еще одну типичную программу для GPU – сложение двух векторов.

Работа с памятью

Память на устройстве выделяется и освобождается CPU с помощью следующих функций (табл. 1).

Таблица 1

CUDA функции для выделения, копирования и освобождения памяти на GPU

Назначение функции	Синтаксис функции
Выделение памяти	<code>cudaMalloc(void **devPtr, size_t size);</code>
Освобождение памяти	<code>cudaFree(void **devPtr);</code>
Копирование данных	<code>cudaMemcpy(void *dst, const void *src, size_t size, enum cudaMemcpyKind kind)</code> В качестве параметра <code>kind</code> , задающего направление копирования, могут выступать: <code>cudaMemcpyHostToDevice</code> (с хоста на устройство), <code>cudaMemcpyDeviceToHost</code> (с устройства на хост), <code>cudaMemcpyDeviceToDevice</code> (с устройства на устройство), <code>cudaMemcpyHostToHost</code> (с хоста на хост).

Простой пример работы функций выделения, копирования и освобождения памяти приведен ниже:

```
float *h_a, *h_b; // host
float *d_a, *d_b; // device
int n = 1024;
int data_sz = n*sizeof(float);

//Выделение памяти на CPU:
h_a = (float *)malloc(data_sz);
h_b = (float *)malloc(data_sz);

//Выделение памяти на GPU:
cudaMalloc((void **) &d_a, data_sz);
cudaMalloc((void **) &d_b, data_sz);

//Инициализация массива на CPU:
for (int i=0; i<n; i++)
h_a[i] = 10.f + i;

//Копирование массива a на GPU:
cudaMemcpy(d_a, h_a, data_sz, cudaMemcpyHostToDevice);

//Копирование массива b на GPU:
cudaMemcpy(d_b, d_a, data_sz, cudaMemcpyDeviceToDevice);

//Копирование массива b на CPU:
cudaMemcpy(h_b, d_b, data_sz, cudaMemcpyDeviceToHost);

//Сравнение результатов:
for(int i=0; i<n; i++)
assert(h_a[i] == h_b[i]);

//Освобождение памяти на CPU и GPU:
free(h_a);
free(h_b);
cudaFree(d_a);
cudaFree(d_b);
```

Сложение двух векторов

Рассмотрим пример сложения двух векторов на GPU:

```
__global__ void sumKernel(float * a, float * b, float
* c)
{
// глобальный индекс нити
int i = threadIdx.x + blockIdx.x * blockDim.x;
c[i] = a[i] + b[i];
}

void sum(float * a, float * b, float * c, int n)
{
int data_sz = n * sizeof(float);
float * d_a;
float * d_b;
float * d_c;

// выделение памяти в GPU
cudaMalloc((void**)&d_a, data_sz);
cudaMalloc((void**)&d_b, data_sz);
cudaMalloc((void**)&d_c, data_sz);

// копирование данных из памяти CPU в память GPU
cudaMemcpy(d_a, a, data_sz, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, b, data_sz, cudaMemcpyHostToDevice);

// вызов ядра для обработки данных
sumKernel <<<dim3(512), dim3(n/512)>>>(d_a, d_b, d_c);

// копирование результатов в память CPU
cudaMemcpy(c, d_c, data_sz, cudaMemcpyDeviceToHost);

// освобождение памяти
cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);
}
```

Первая функция `sumKernel()` является ядром, выполняющим сложение векторов. Спецификатор `__global__` указывает на то, что функция вызывается из CPU и выполняется на GPU. Ядро при помощи встроенных пе-

ременных вычисляет глобальный индекс нити, для которого выполняется сложение, и суммирует соответствующие элементы. Функция `sum()` выделяет память на GPU, копирует входные данные из памяти на CPU в память GPU, вызывает ядро `sumKernel()`, копирует результат обратно в память CPU и освобождает выделенную память.

Типичная структура **CUDA** программы

В приведенном выше примере хорошо показаны этапы вызова CUDA функций. Как правило, типичная структура CUDA программы имеет следующий вид:

1. Объявление глобальных переменных;
2. Описание прототипов функций:
 - `__global__ void kernel();`
3. Функция `main()`:
 - Размещение памяти на устройстве `cudaMalloc();`
 - Передача данных с процессора на устройство `cudaMemcpy();`
 - Определение параметров запуска ядра;
 - Вызов ядра `kernel<<<block, thread>>>();`
 - Передача результатов с устройства на процессор `cudaMemcpy();`
 - Дополнительно: сравнение результатов расчета на процессоре и устройстве;
4. Ядро `__global__ void kernel()`:
 - Объявление переменных `local, shared;`
 - Синхронизация `__syncthreads();`

Обработка ошибок

После выполнения функций на GPU, они возвращают значения типа `cudaError_t`. Функции, успешно отработавшие на девайсе, возвращают значение `cudaSuccess`, в противном случае возвращается код ошибки. **Таблица?**

Назначение функции	Синтаксис функции
Возвращает описание ошибки в виде строки	<code>cudaGetErrorString(cudaError_t error);</code>
Возвращает код последней ошибки	<code>cudaGetLastError();</code>

Основные выводы ответа при обработке ошибок:

- 1) `cudaSuccess` – при удачном выполнении;
- 2) `cudaErrorMemoryAllocation` – при ошибке выделения памяти;
- 3) `cudaErrorInvalidValue` – неверные параметры аргумента (например, размер копирования отрицателен);
- 4) `cudaErrorInvalidDevicePointer` – неверный указатель памяти в видеокарте;
- 5) `cudaErrorInvalidMemcpyDirection` – неверное направление (например, перепутан источник и место назначения копирования);
- 6) `cudaErrorInitializationError` – ошибка инициализации;
- 7) `cudaErrorPriorLaunchFailure` – ошибка при предыдущем асинхронном запуске функции;
- 8) `cudaErrorInvalidResourceHandle` – неверный обработчик события.

Приведем пример использования функции `cudaGetLastError()`.

```
cudaError_t err = cudaGetLastError();
if (err != cudaSuccess) printf("%s ", cudaGetErrorString(err));
```

Здесь проверяется, есть ли ошибка у последней команды, выполненной на девайсе. Если ошибка есть, то выводится ее описание.

Замер времени работы части кода программы

Одним из важных параметров при написании программы является время, затраченное на ее выполнение. Для замера времени выполнения операций на CPU есть множество возможностей. Для того, чтобы замерить время, которое было затрачено на выполнение вычислений на GPU, можно воспользоваться

механизмом событий (CUDA events). Событие – это объект типа `cudaEvent_t`, используемый для обозначения «точки» среды вызовов CUDA.

Приведем пример фрагмента кода, измеряющего время выполнения ядра на GPU:

```
// инициализируем события
cudaEvent_t start, stop;
float elapsedTime;

// создаем события
cudaEventCreate(&start);
cudaEventCreate(&stop);

// запись события
cudaEventRecord(start, 0);

// вызов ядра
cudaEventRecord(stop, 0);

// ожидание завершения работы ядра
cudaEventSynchronize(stop);
cudaEventElapsedTime(&elapsedTime, start, stop);

// вывод информации
printf("Time spent executing by the GPU: %.2f
milliseconds\n", elapsedTime);

// уничтожение события
cudaEventDestroy(start);
cudaEventDestroy(stop);
```

Синхронизация

Для синхронизации текущей нити на CPU с GPU используется функция `cudaThreadSynchronize()`, которая дожидается завершения выполнения всех операций CUDA, ранее вызванных с данной нити CPU. Функция CPU `cudaDeviceSynchronize()` блокирует выполнение кода CPU, пока все GPU ядра

не выполнены. Для синхронизации всех нитей в одном и том же блоке используется функция `__syncthreads()`, которая блокирует вызывающие нити до тех пор, пока все нити блока не войдут в эту функцию. При помощи этой функции можно организовать барьеры внутри ядра, гарантирующие, что если хотя бы одна нить прошла такой барьер, то не осталось ни одной за барьером (не прошедшей его). Такая синхронизация называется барьерной (рис. 1).

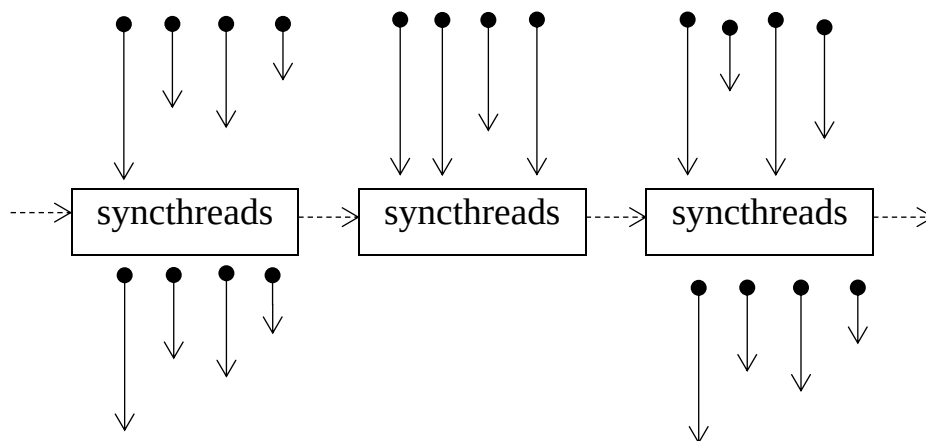


Рис. 1. Барьерная синхронизация

Нежелательно размещать эту команду внутри условного оператора `if`, а также следует обеспечивать безусловный вызов этой функции всеми потоками блока.

Выпишем остальные «барьерные команды»:

- **`__threadfence_block()`** заставляет ждать вызвавший её поток, пока все совершенные операции доступа к разделяемой и глобальной памяти завершатся и станут видны потокам этого блока.
- **`__threadfence()`** заставляет ждать вызвавший её поток, пока все совершенные операции доступа к разделяемой памяти станут видны потокам этого блока, а операции с глобальной памятью – всем потокам на девайсе.
- **`__threadfence_system()`** подобна `__threadfence()`, но включает синхронизацию с потоками на CPU («хосте»), при использовании page-locked памяти.

Практическая часть

1. Постройте зависимость скорости копирования данных от объема для одномерного массива с CPU на GPU и наоборот.
2. Измерьте время выполнения кода функции сложения двух векторов с помощью функций C и cudaEvent. Объясните разницу.

Контрольные вопросы

1. Перечислите виды синхронизации.
2. Какие основные ошибки возникают при исполнении кода на CUDA?
3. Опишите типичную структуру CUDA программы.